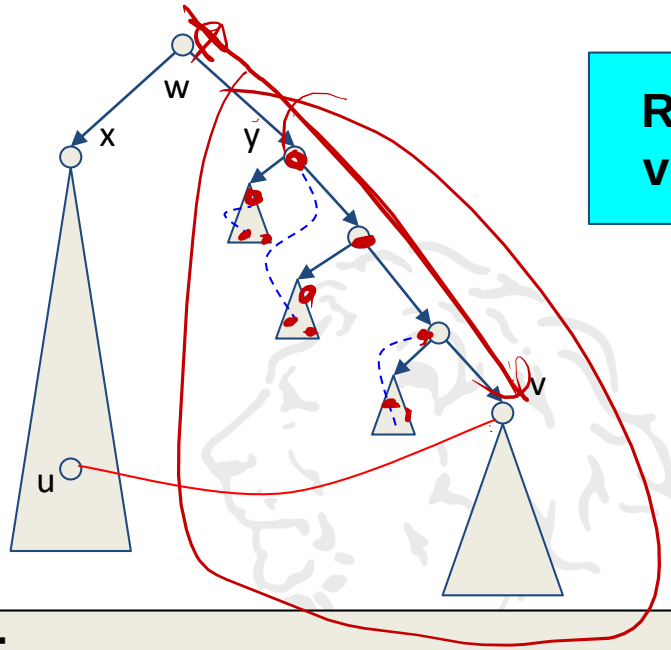
Dynamic Graph Algorithms

CSC-531

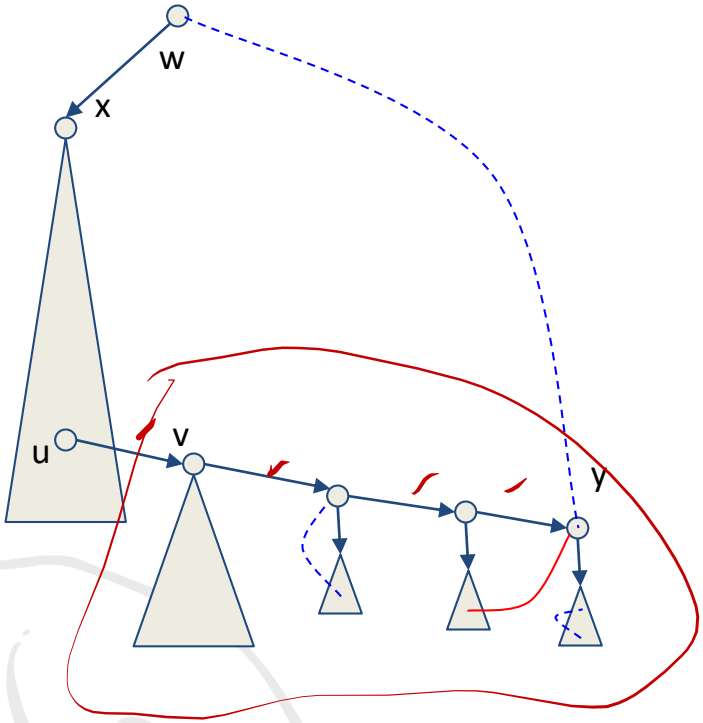
Incremental DFS in Undirected Graphs

Dr. Shahbaz Khan
Department of Computer Science and Engineering,
Indian Institute of Technology Roorkee
shahbaz.khan@cs.iitr.ac.in

Incremental Depth First Search Trees (Undirected)



Reverse v-y path



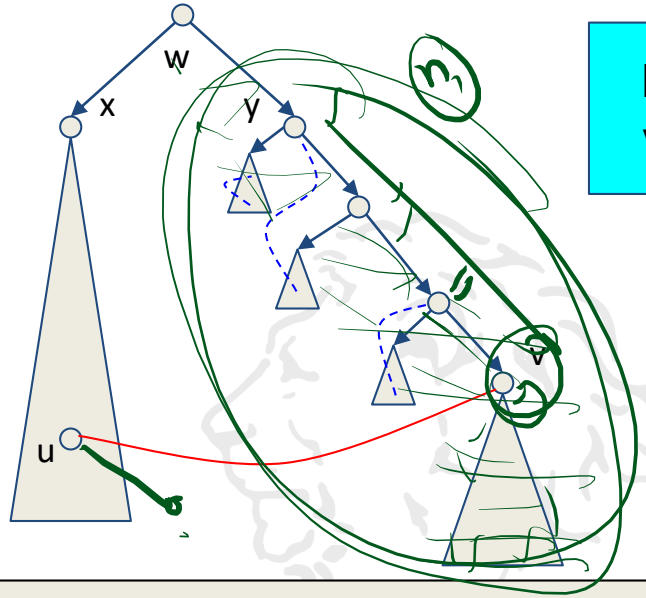
Algorithm:

InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G // lev(u)<=lev(v)
    If(LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure → O(T(v))

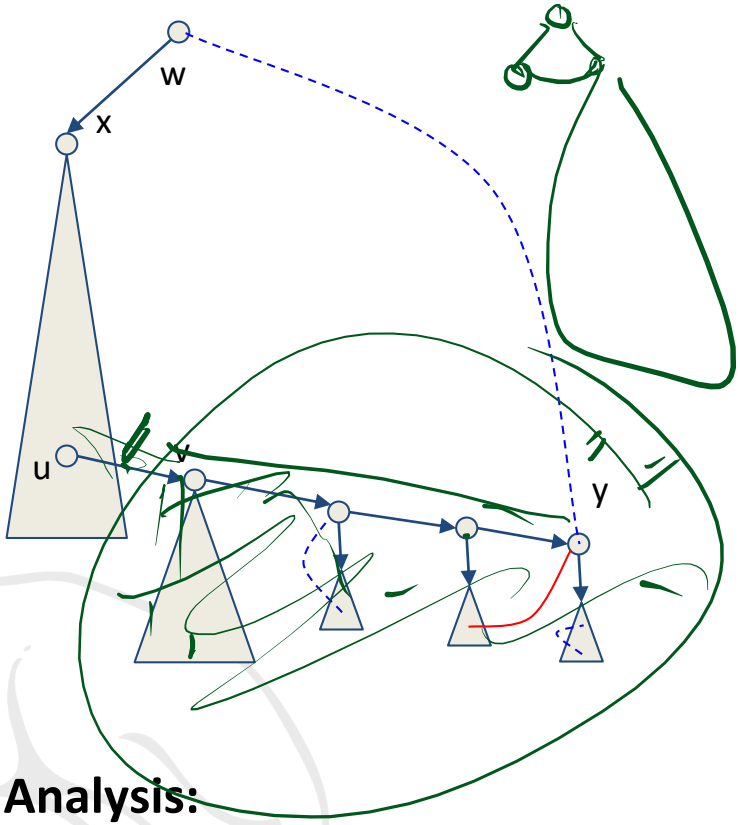
Analysis?

Collect all Non-tree edges in InsE

Incremental Depth First Search Trees (Undirected)



Reverse v-y path



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

O(1)

Analysis:

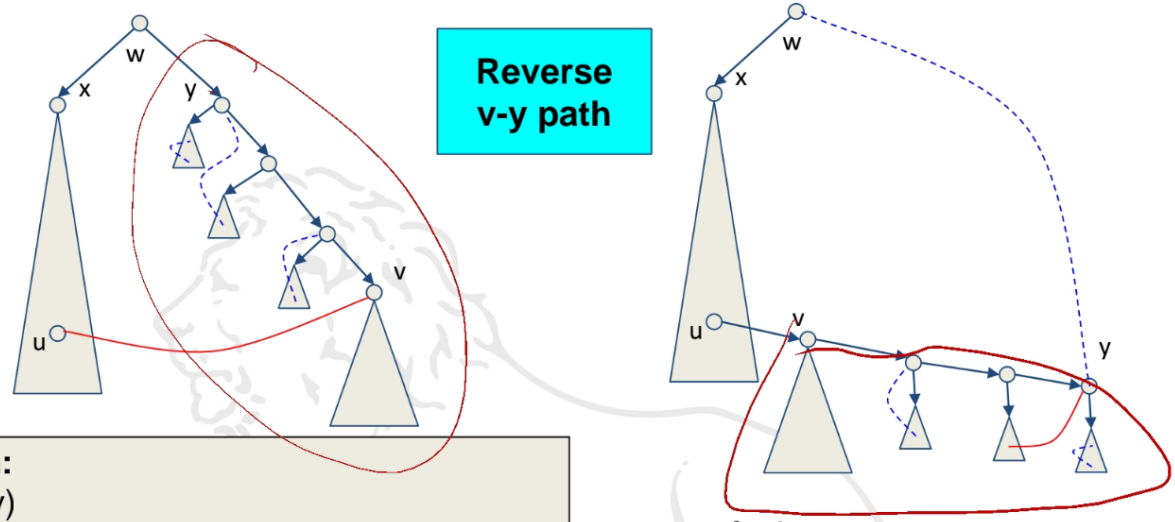
deg(x) whenever x in T(v)

Or deg(x) whenever x falls

WC = O(m+n)

Update LCA structure

Incremental Depth First Search Trees (Undirected)



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

Analysis:

deg(x) whenever x in T(v)

Or deg(x) whenever x falls

Total= O(n) ∑ deg(v) = O(mn)

Amortized O(n)

Incremental Depth First Search Trees (Undirected)

Suppose no-cross edges needed to be collected

↳ Cost is $|T(y)|$

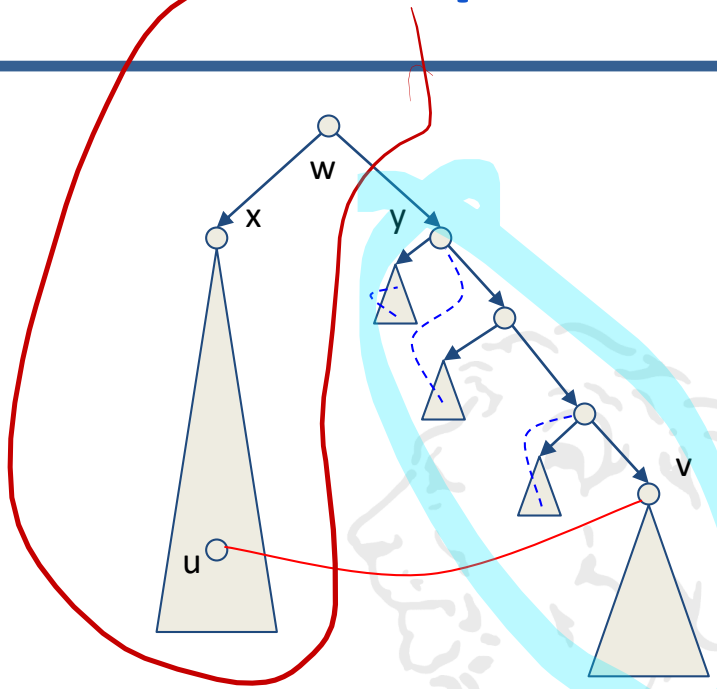
WC $O(n)$

Am. $O(n^2/m)$ → Dense $O(1)$

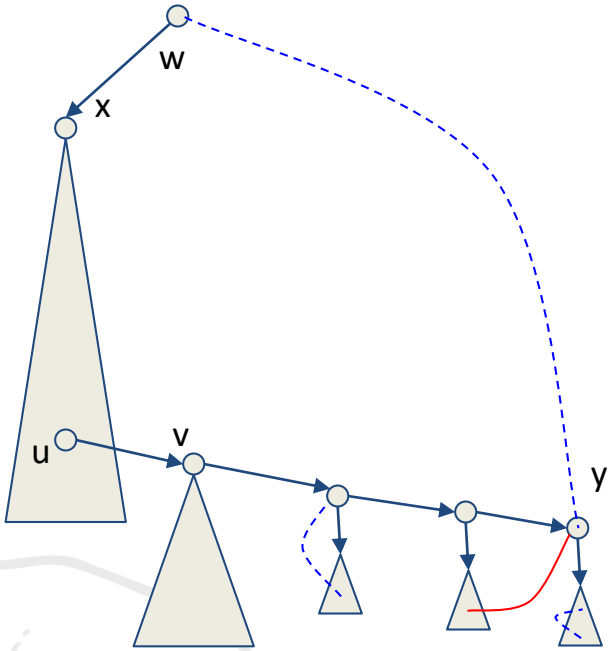
Total $O(n) \times \sum_{v \in V} 1$

$= O(n^2)$

Incremental Depth First Search Trees (Undirected)



Reverse v-y path



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

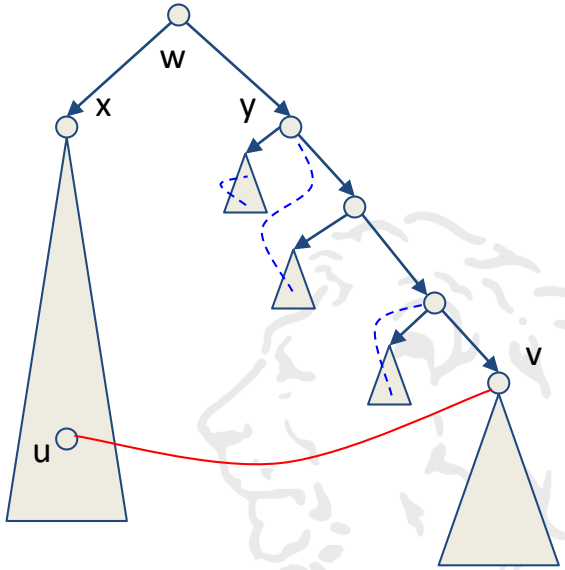
Correctness?

Every non-tree edge is a back edge

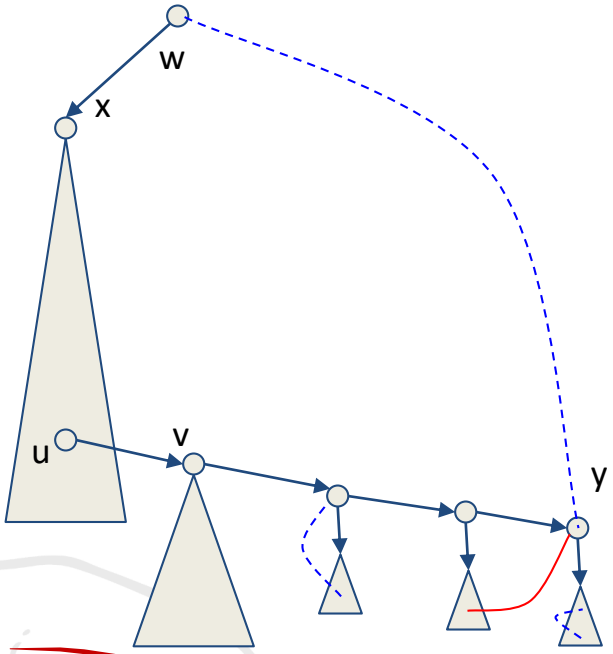
Induction

Inv G \ InsE has no cross edge

Incremental Depth First Search Trees (Undirected)

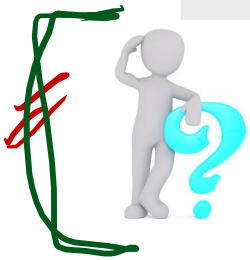


Reverse v-y path



Approach: O(mn)
Collect the Cross Edges
Insert them one by one

Improve to O(n2)



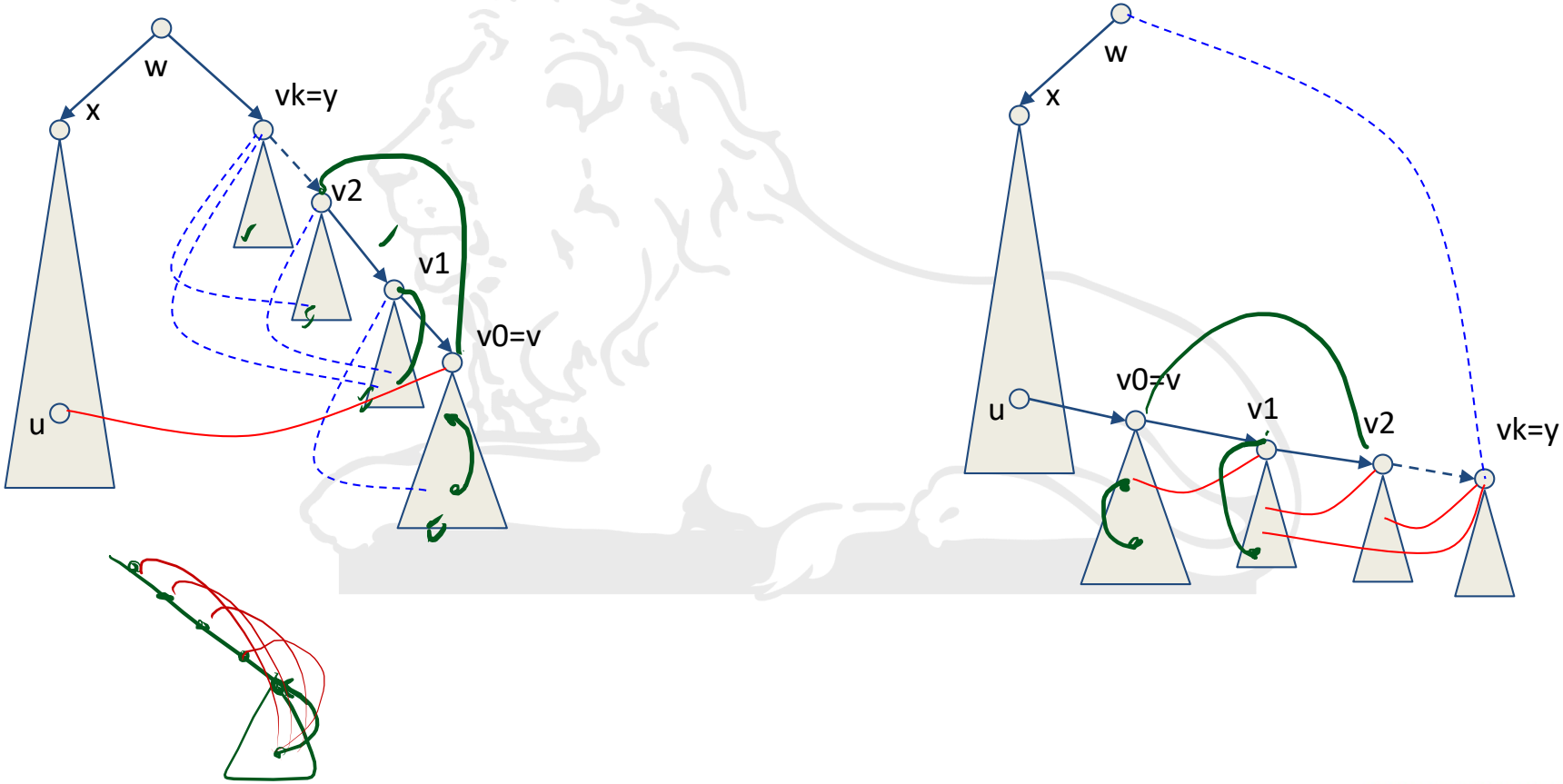
Think?
How does this cross edge look like?
How many such cross edges?

Back Edges converting to Cross Edge

Think?

How does this cross edge look like?

How many such cross edges?

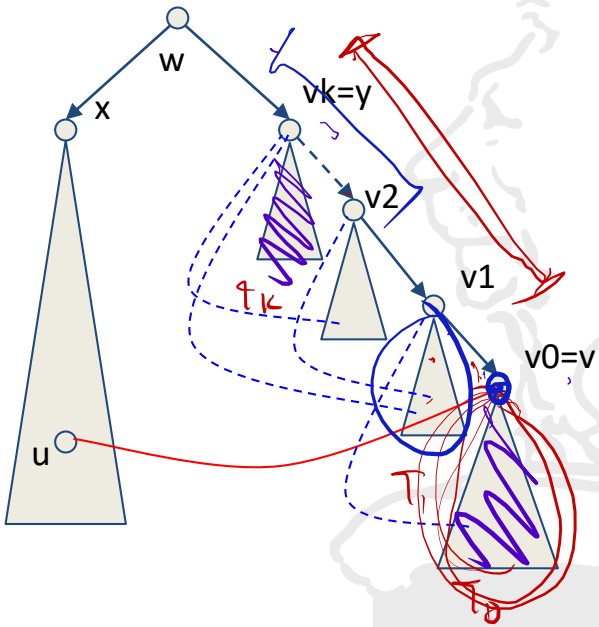


Back Edges converting to Cross Edge

Think?

How does this cross edge look like?

How many such cross edges?



Edges between Ti and vj, j>i

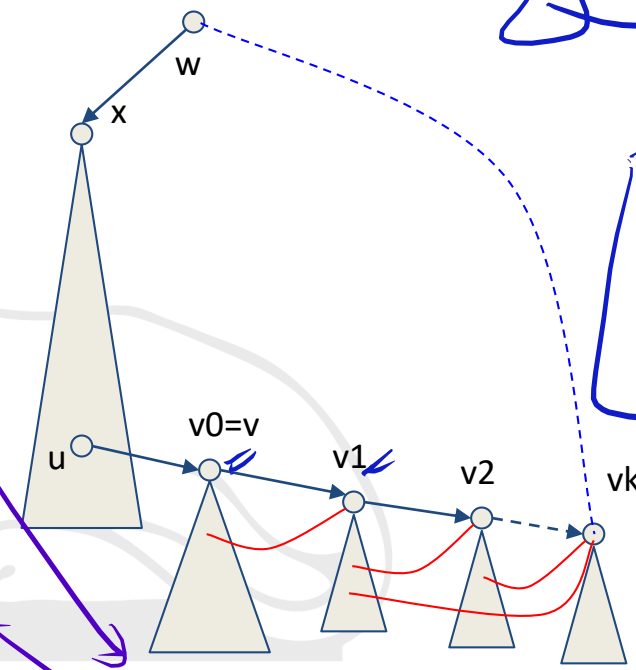
Number of edges

$\sum |T_i| (k-i)$

Fall of vertices

$\sum |T_i| (i)$

$\times (2i+1)$

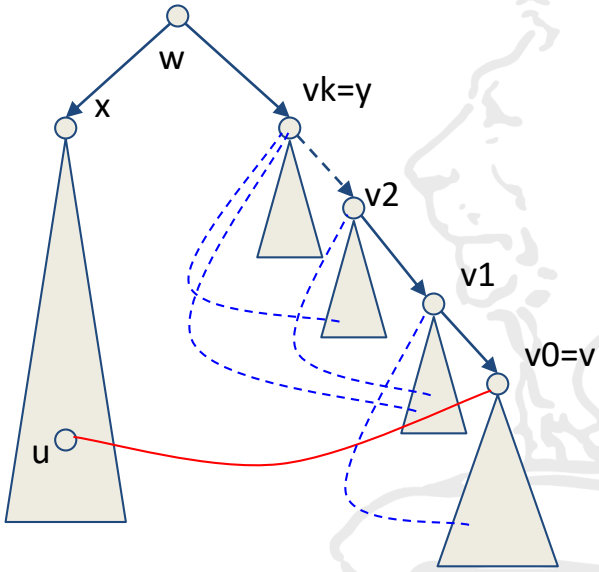


Back Edges converting to Cross Edge

Think?

How does this cross edge look like?

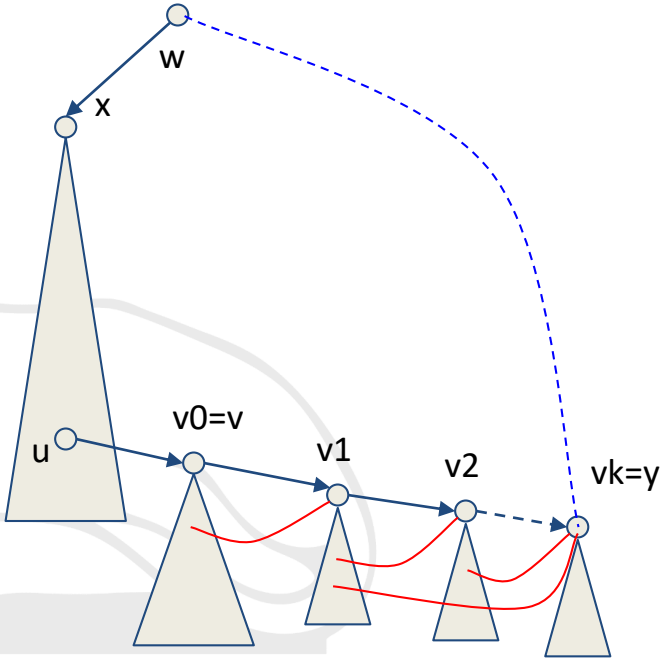
How many such cross edges?



Edges between Ti and vj, j>i

Number of edges ∑ |Ti| (k-i)

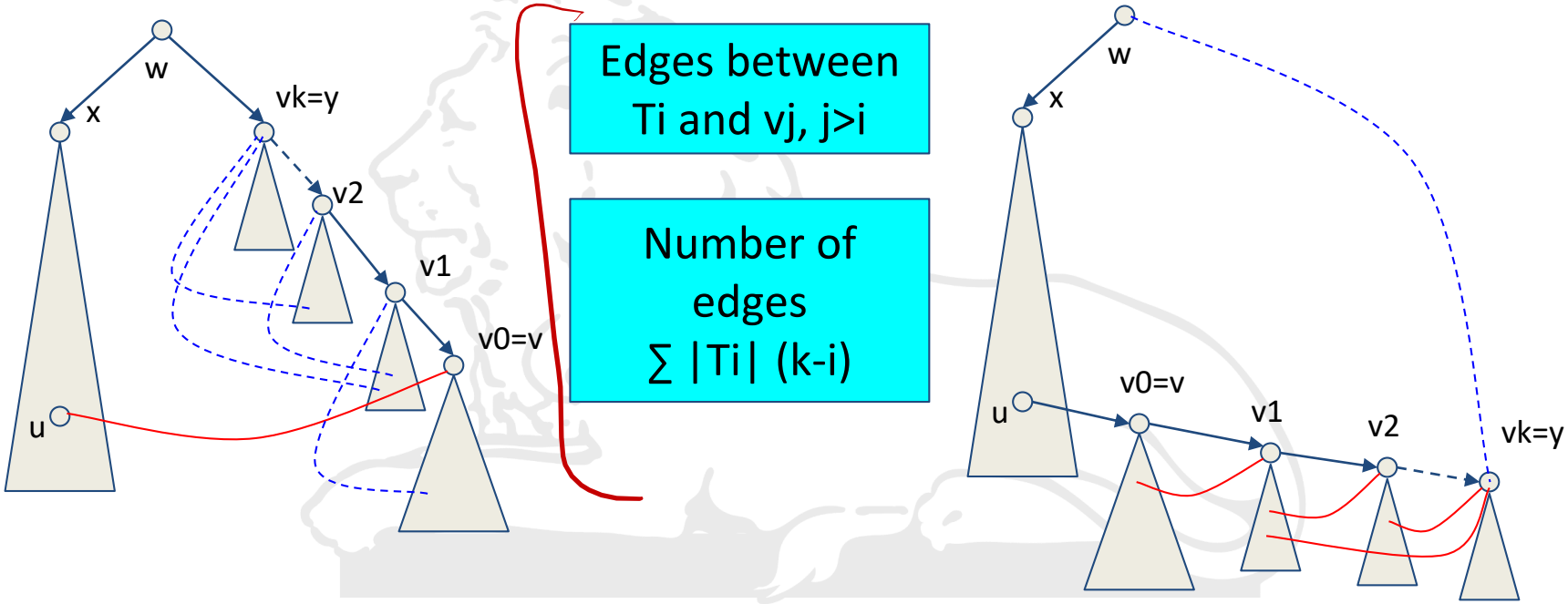
Fall of vertices ∑ |Ti| (i)



Processing **Cross Edge** Good because it leads to Fall

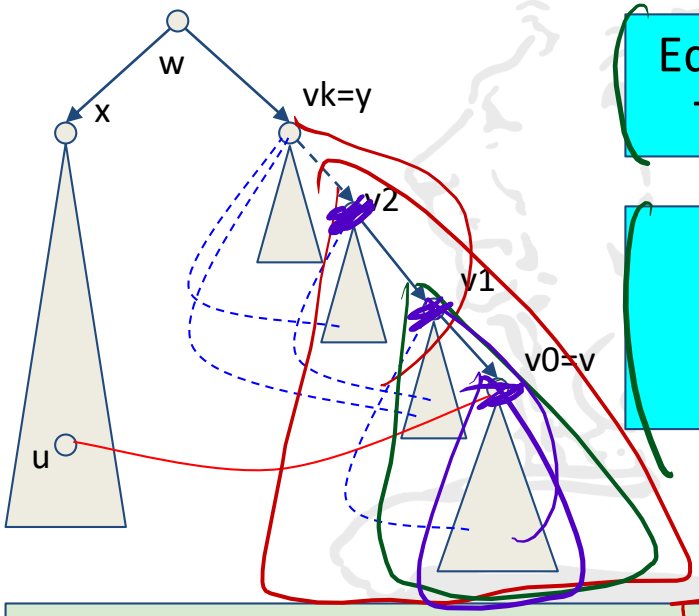
Back Edges converting to Cross Edge

Think?
How to efficiently collect such edges?



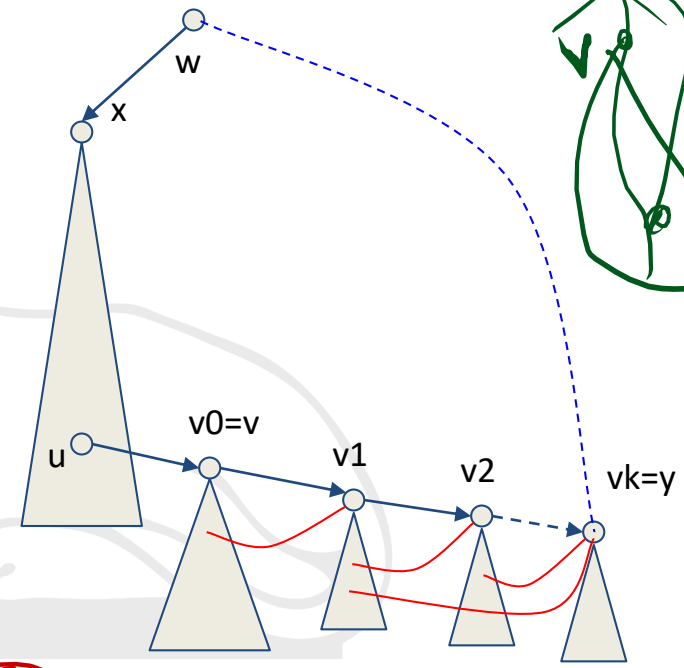
Back Edges converting to Cross Edge

Think?
How to efficiently collect such edges?



Edges between Ti and vj, j>i

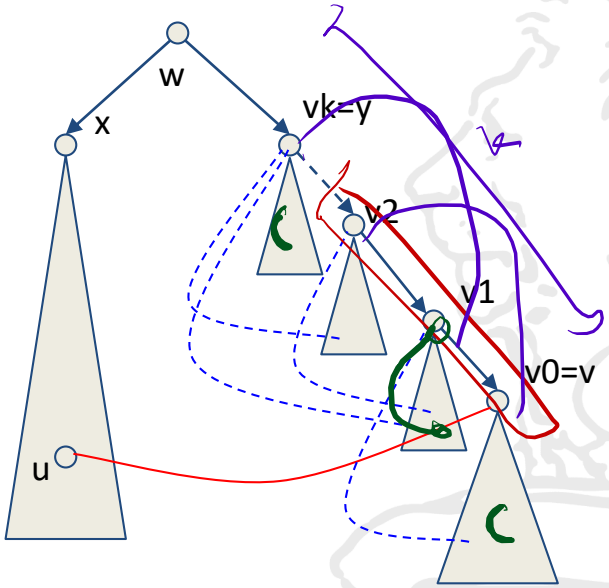
Number of edges
∑ |Ti| (k-i)



Each vertex v store back edges B[v]
- From the subtree T(v)
- To parent of v

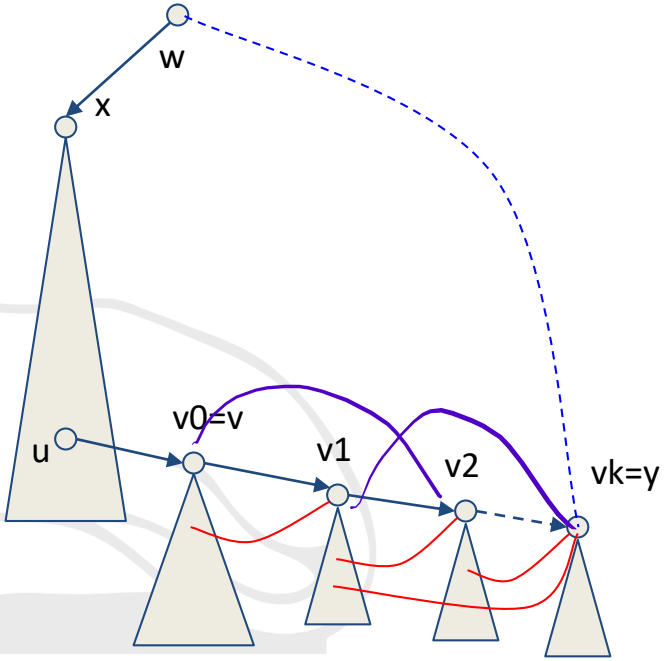
Back Edges converting to Cross Edge

Think?
How to efficiently collect such edges?



Edges between Ti and vj, j>i

Number of edges
∑ |Ti| (k-i)



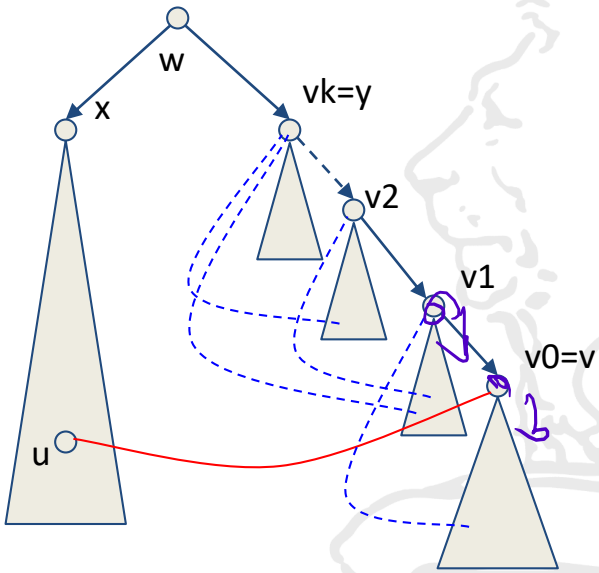
Each vertex v store back edges **B[v]**
- From the subtree T(v)
- To parent of v

REQUIRED EDGES: B[x] for x in v->y {y}

Problem?

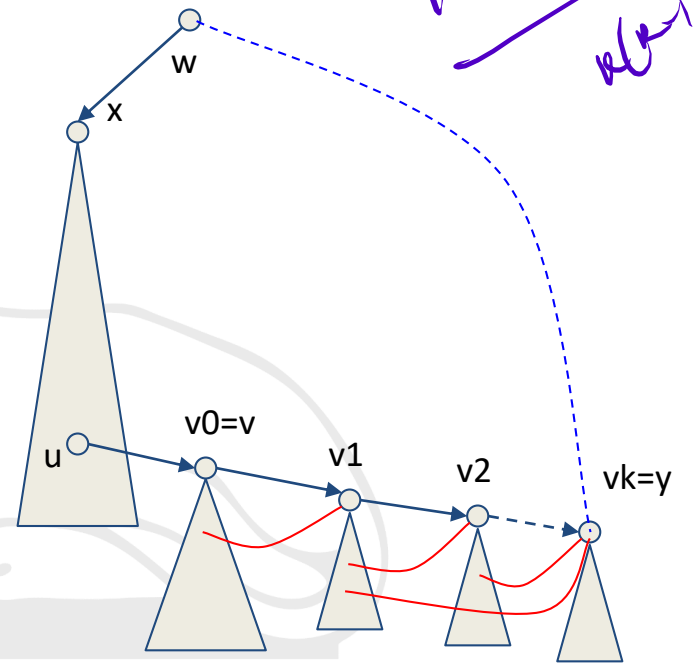
Back Edges converting to Cross Edge

Think?
How to efficiently collect such edges?



Edges between Ti and vj, j>i

Number of edges
∑ |Ti| (k-i)

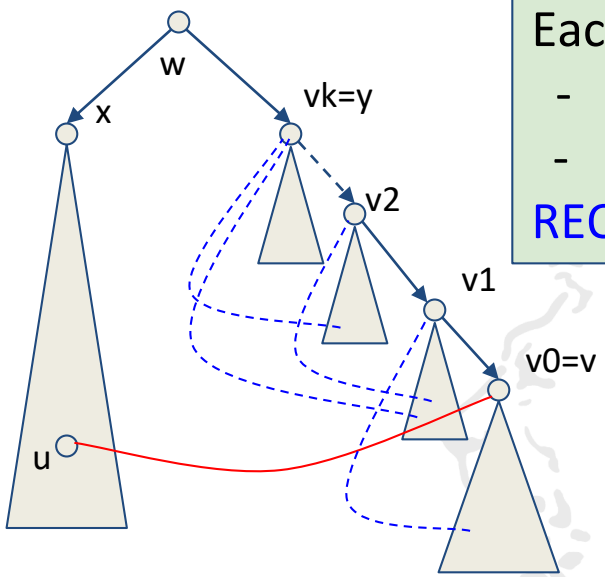


Each vertex v store back edges **B[v]**
- From the subtree T(v)
- To parent of v

REQUIRED EDGES: B[x] for x in v->y

Problem?
k(k-1) back edges collected from v->y

Incremental Depth First Search Trees (Undirected)



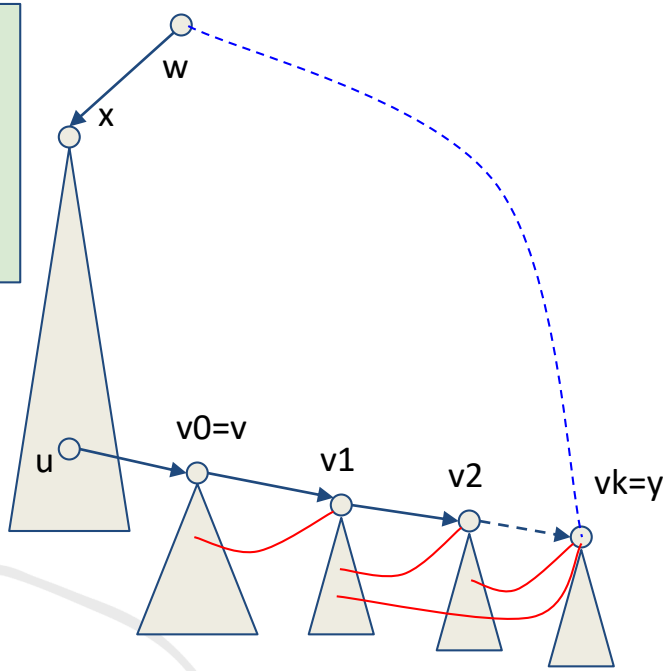
Each v store back edges **B[v]**

- From the subtree T(v)
- To parent of v

REQUIRED: B[x] for x in v->y

Problem?

k(k-1) back edges collected from v->y



Target Analysis:

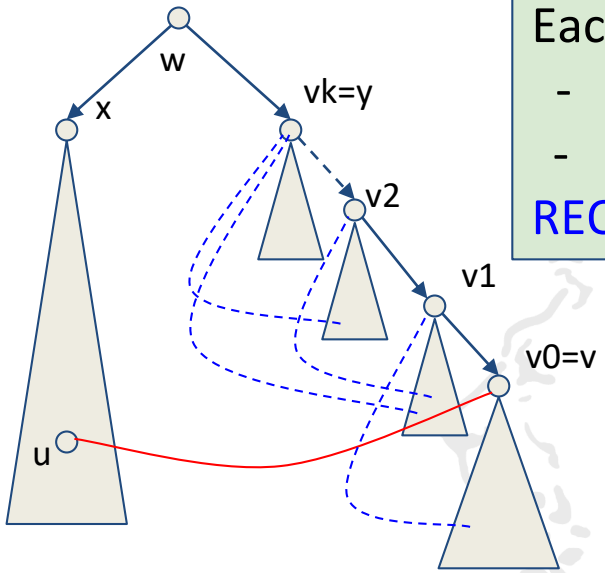
Charge O(1) whenever x falls

Total= n x O(n) = O(n2)

Processing **Cross Edge** Good because it leads to Fall

What about Back Edges?

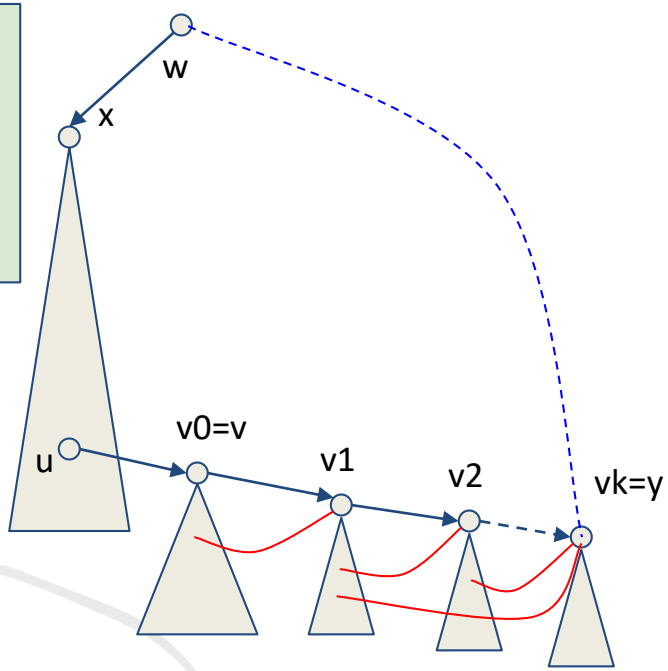
Incremental Depth First Search Trees (Undirected)



Each v store back edges **B[v]**
- From the subtree T(v)
- To parent of v

REQUIRED: B[x] for x in v->y

Problem?
k(k-1) back edges
collected from v->y



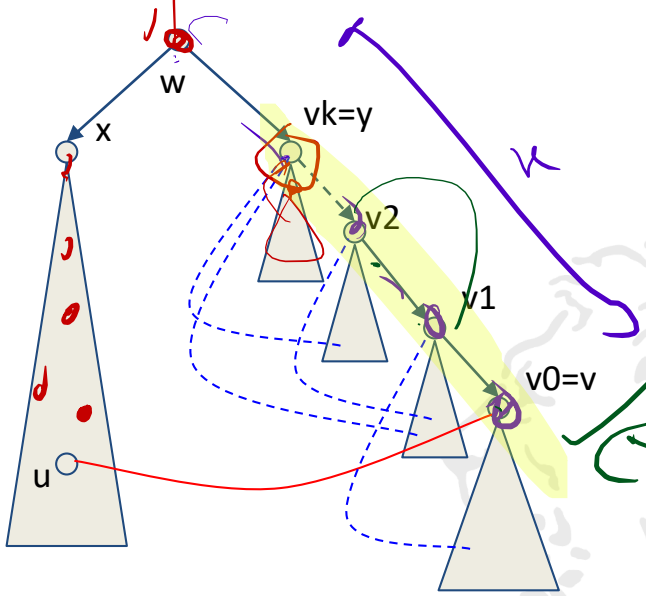
Target Analysis:
Charge O(1) whenever x falls
Total= n x O(n) = O(n2)

Processing **Cross Edge** Good
because it leads to Fall
What about Back Edges?

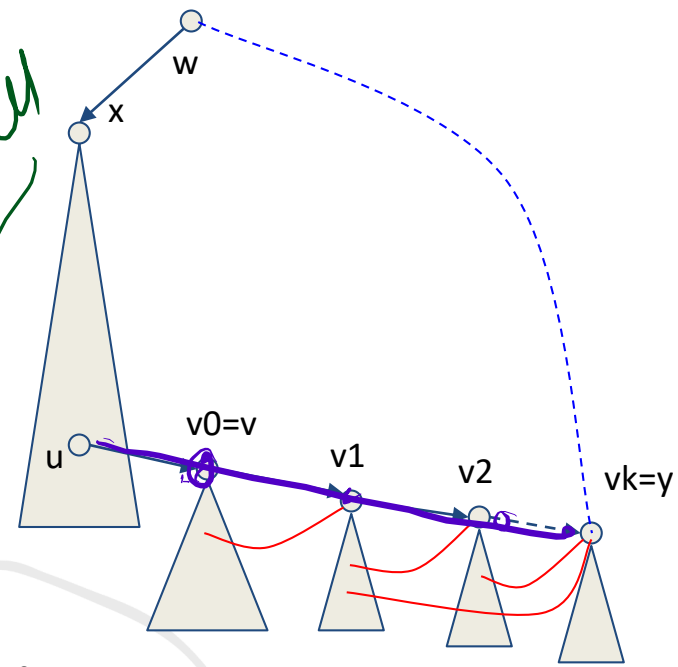
Minimum Fall? v->y path

1+2+...+k = k(k-1) levels

Incremental Depth First Search Trees (Undirected)



Update B[·] in some time to collect all



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

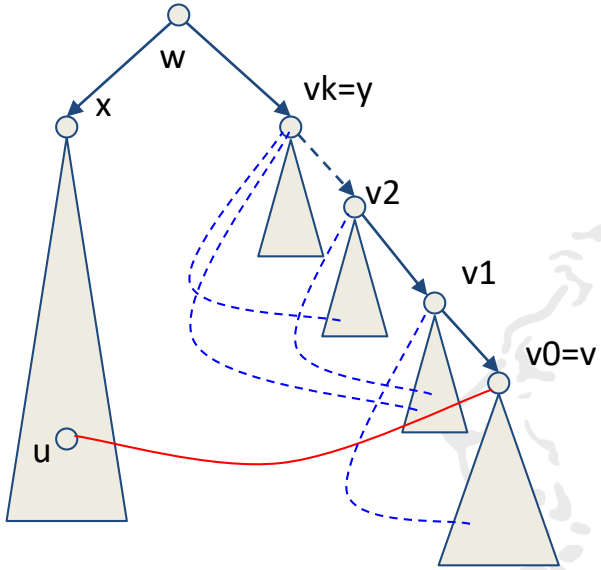
Analysis:

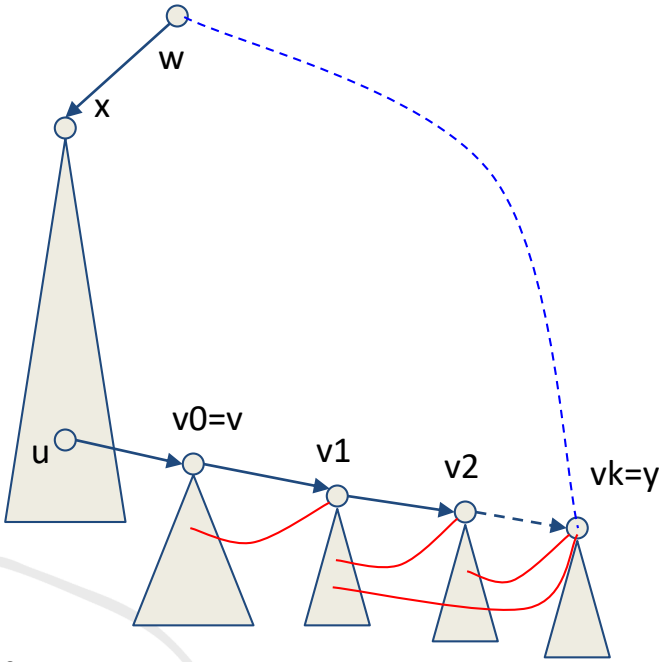
Charge O(1) whenever x falls

Total= n x O(n) = O(n2)

- Traverse and Reverse Path
- Collect Back Edges
- Collect Cross Edges => Fall

Incremental Depth First Search Trees (Undirected)





Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
        (u,v) = Pop from InsE
        Add (u,v) to G          // lev(u)<=lev(v)
        If( LCA(u,v)!={u,v})
                Reverse (v,LCA(u,v)) except LCA(u,v)
                Add (u,v) to DFS Tree
                Add cross edges of T(v) in InsE
                Update LCA structure
```

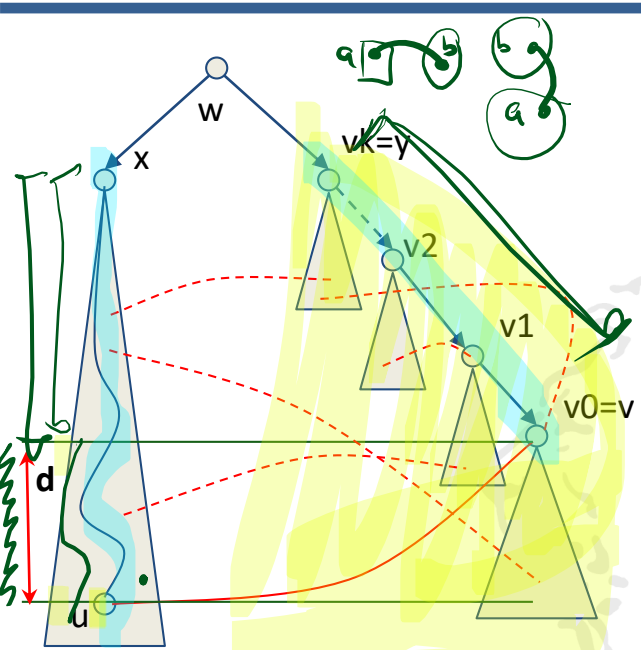
Analysis:

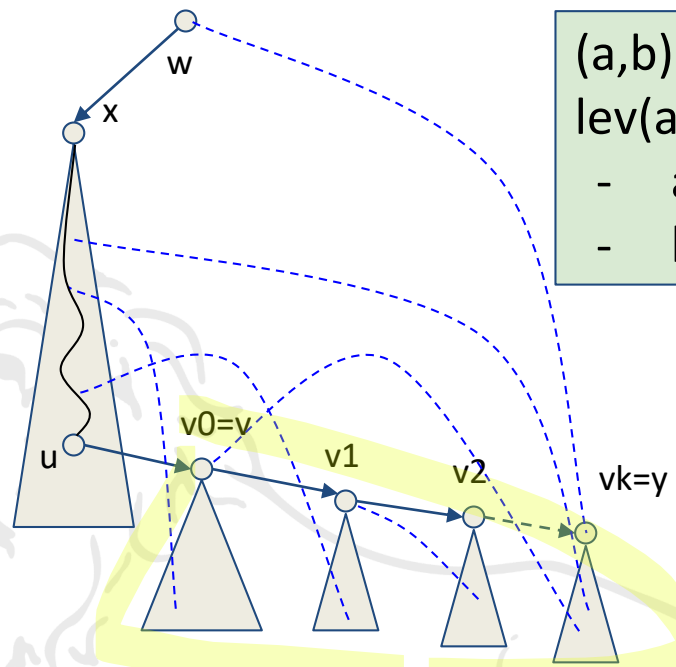
Charge O(1) whenever x falls

Total= n x O(n) = O(n2)

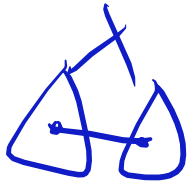
- Traverse and Reverse Path
- Collect Back Edges
- Collect Cross Edges => Fall
- Collect Cross Edge => No fall?

Cross Edges Converting to Back Edges?





(a,b) converted?
lev(a)<lev(b)
- a is new ancestor of b
- b is new descendant of a



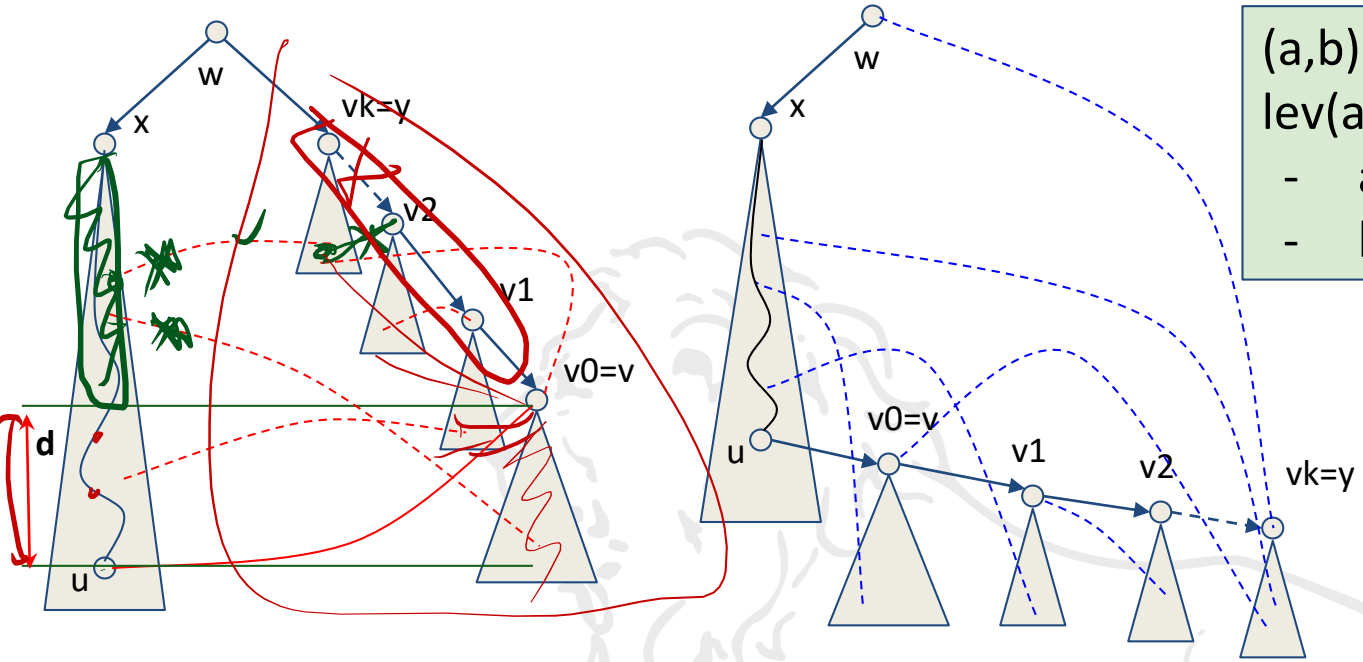
During above insertion

- Which vertices got new descendants? x->u and v->y
- Which vertices got new ancestors? T(v)
- Maximum conversions of cross edges? (|x->u|+|v->y|) * |T(v)|
- Minimum Fall in vertices?

(1+d)* |T(v)|

Minimum Fall >= **Maximum Conversions**?

Cross Edges Converting to Back Edges?



(a,b) converted?
lev(a)<lev(b)
- a is new ancestor of b
- b is new descendant of a

IDEA
Process cross edges with highest end vertices first?

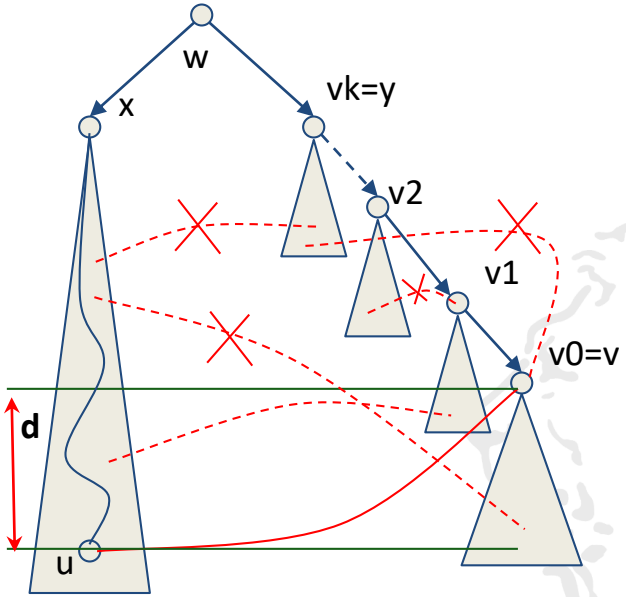
During above insertion

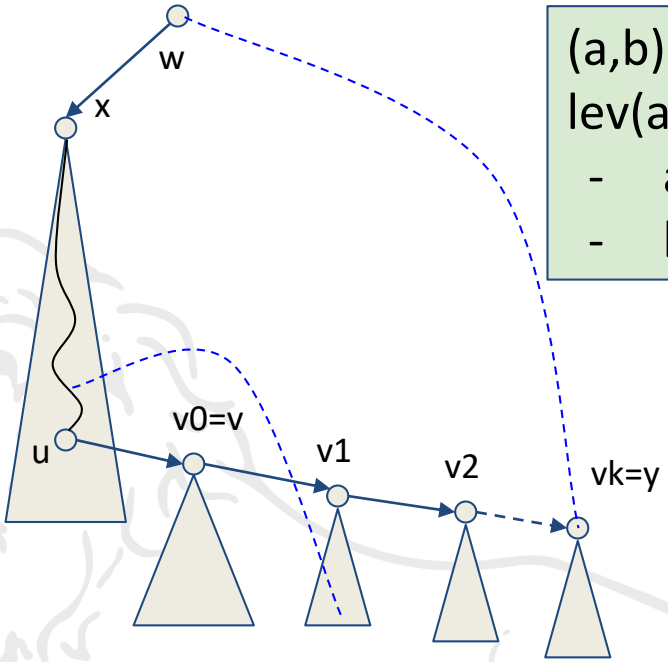
- Which vertices got new descendants? x->u and v->y
- Which vertices got new ancestors? T(v)
- Maximum conversions of cross edges? $\underbrace{(|x\text{->}u|+|v\text{->}y|) * |T(v)|}_{(d+1)}$
- Minimum Fall in vertices?

$(1+d) * |T(v)|$

Minimum Fall >= **Maximum Conversions**?

Cross Edges Converting to Back Edges?





(a,b) converted?
lev(a)<lev(b)
- a is new ancestor of b
- b is new descendant of a

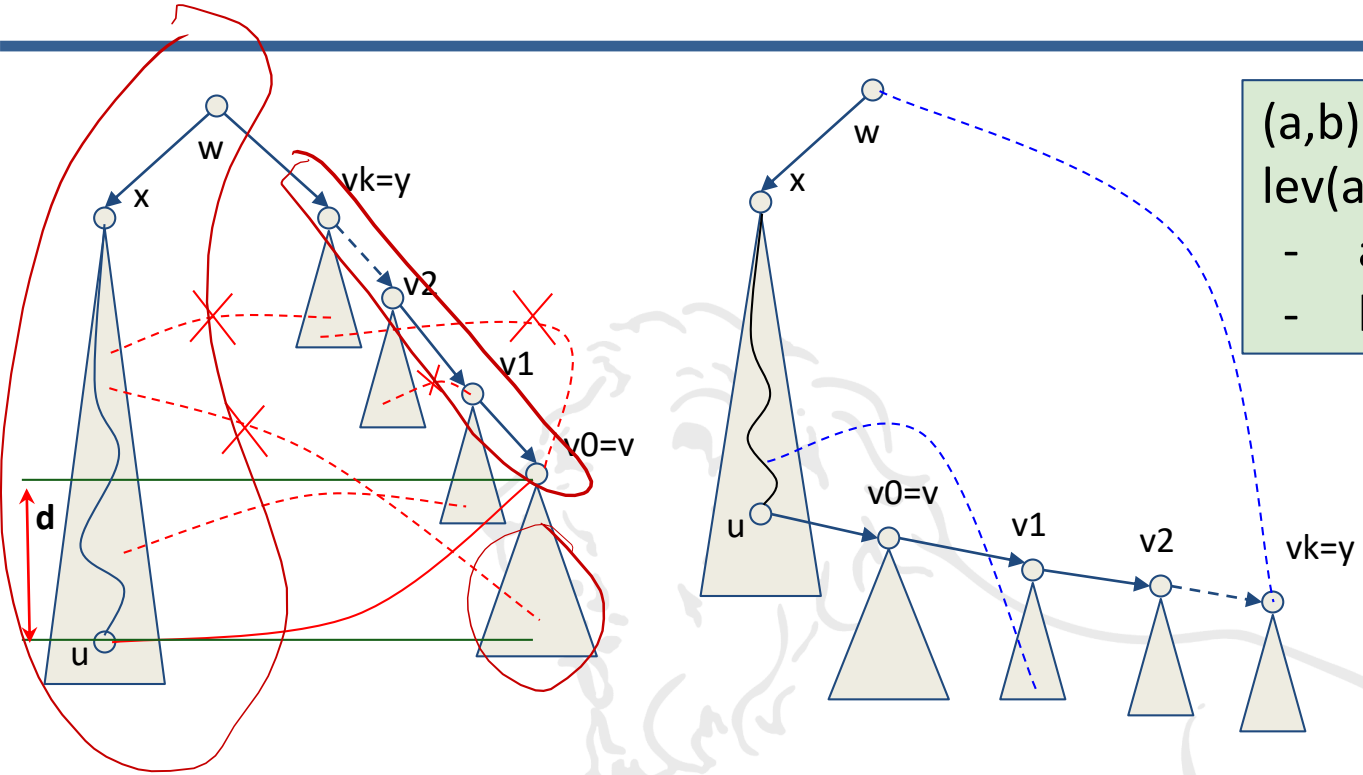
IDEA
Process cross edges with highest end vertices first?

During above insertion

- Which vertices got new descendants? ~~x->u and v->y~~ u'->u and y?
- Which vertices got new ancestors? T(v)
- Maximum conversions of cross edges?
- Minimum Fall in vertices?

Minimum Fall >= **Maximum Conversions**?

Cross Edges Converting to Back Edges?



(a,b) converted?
lev(a)<lev(b)
- a is new ancestor of b
- b is new descendant of a

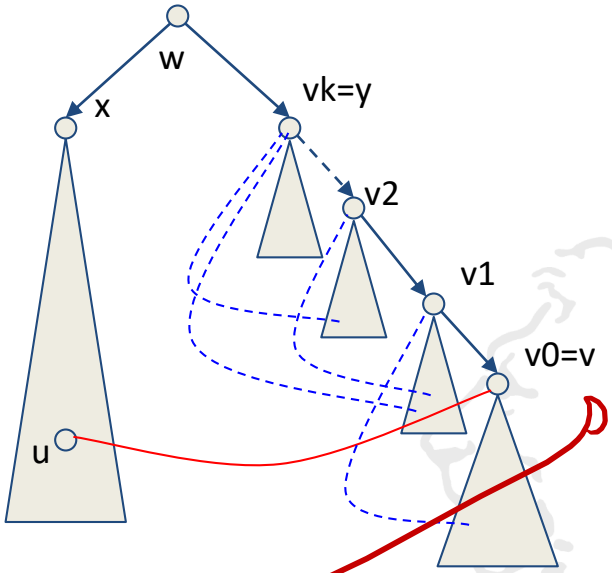
IDEA
Process cross edges with highest end vertices first?

During above insertion

- Which vertices got new descendants? ~~x->u and v->y~~ u'->u and y?
- Which vertices got new ancestors? T(v)
- Maximum conversions of cross edges? d * |T(v)| + | T(v1)|?
- Minimum Fall in vertices? d * |T(v)| + |T(v1)|?

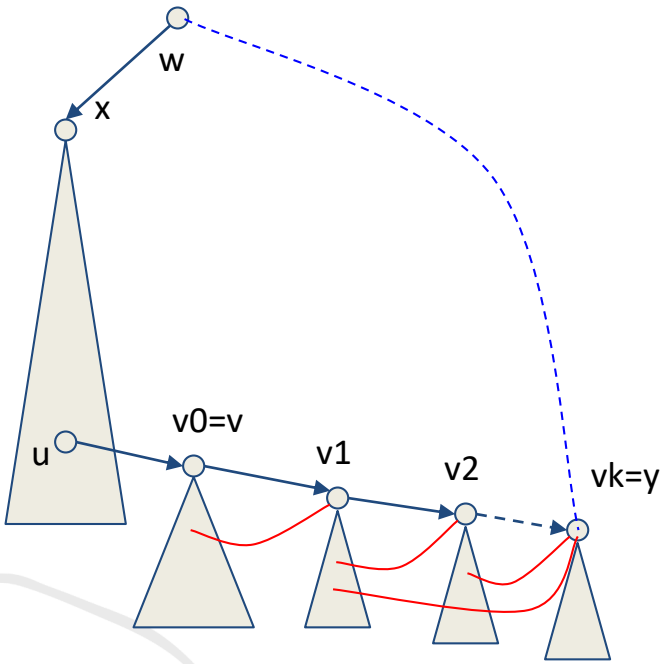
Minimum Fall >= **Maximum Conversions**?

Incremental Depth First Search Trees (Undirected)



IDEA
Process cross edges with highest end vertices first?

elu store in B[•]



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop highest from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

Analysis:

Charge O(1) whenever x falls

Total= n x O(n) = O(n2)

- Traverse and Reverse Path
- Collect Back Edges
- Collect Cross Edges => Fall
- Collect Cross Edge => No fall

Question

Think about **incremental maintenance of DFS** trees.

Directed acyclic graph **O(mn)**

Undirected **O(n2)**

Directed graph **O(m2)**

Paper Reference

Baswana Khan 14

Decremental DFS in DAGs
Ponder over how DFS of a DAG changes, on deleting an edge?